

# LlamaCloud Image Extraction — Code Explained

A walk-through of the Python script that downloads images from a LlamaCloud parse result and writes a JSON manifest describing them.

## Source Code

```
image_records = []

images_metadata = result.images_content_metadata

if images_metadata:
    print(
        "\nTotal images available:",
        images_metadata.total_count
    )

    for image in images_metadata.images:
        filename = safe_filename(image.filename)
        category = getattr(
            image,
            "category",
            None
        )

        destination_directory = (
            get_image_output_directory(category)
        )

        destination_path = (
            destination_directory / filename
        )

        download_success = False

        if image.presigned_url:
            download_success = download_file(
                image.presigned_url,
                destination_path
            )

        image_record = {
            "index": image.index,
            "filename": filename,
            "category": category,
            "content_type": getattr(
                image,
                "content_type",
                None
            ),
            "bbox": object_to_dict(
                getattr(image, "bbox", None)
            ),
            "local_path": (
                str(destination_path)
                if download_success
                else None
            ),
        },
```

```

        "download_success": download_success,
    }

    image_records.append(image_record)

    print(
        f"{category or 'other'}: "
        f"{filename} → "
        f"{'saved' if download_success else 'failed'}"
    )

image_records_path = OUTPUT_DIR / "image_records.json"

with image_records_path.open(
    "w",
    encoding="utf-8"
) as file:
    json.dump(
        image_records,
        file,
        indent=2,
        ensure_ascii=False,
        default=str
    )

print("Image records saved:", image_records_path)

```

## Line-by-Line Explanation

```
image_records = []
```

Initialize an empty list to collect metadata dicts — one per image. Like `var imageRecords = new List<ImageRecord>();` in C#.

```
images_metadata = result.images_content_metadata
```

Pull the images section out of the LlamaCloud parse result. This is a container object holding a count plus a list of image descriptors.

```
if images_metadata:
```

Guard clause — if the document had no images, `images_metadata` is `None` or falsy, and we skip the whole block. Same as `if (imagesMetadata is not null)`.

```
print("\nTotal images available:", images_metadata.total_count)
```

Log how many images the parser found. Purely diagnostic.

```
for image in images_metadata.images:
```

Iterate each image descriptor. `images` is a list of objects with fields like `filename`, `presigned_url`, `bbox`, etc.

```
filename = safe_filename(image.filename)
```

Sanitize the filename — strips path separators, illegal characters, whatever your `safe_filename` helper does. Prevents path traversal (`../..`/etc/passwd) and OS-invalid characters.

```
category = getattr(image, "category", None)
```

Read `image.category` if it exists, else `None`. `getattr` with a default is defensive — some image types may not have a category field. Equivalent to `image?.Category` in C#.

```
destination_directory = get_image_output_directory(category)
```

Pick the folder for this category — e.g. `output/charts/`, `output/figures/`, `output/tables/`. Your helper likely creates the directory if missing.

```
destination_path = destination_directory / filename
```

Build the full path using `pathlib.Path`'s `/` operator (path join, not division). Cross-platform equivalent of `Path.Combine(destinationDirectory, filename)`.

```
download_success = False
```

Default the flag to false. Only flips to true if the download actually works.

```
if image.presigned_url:
    download_success = download_file(image.presigned_url, destination_path)
```

If LlamaCloud gave us a URL, download the bytes to disk. `download_file` returns a bool. Skipping the download when no URL exists prevents a crash on `None`.

```

image_record = {
    "index": image.index,
    "filename": filename,
    "category": category,
    "content_type": getattr(image, "content_type", None),
    "bbox": object_to_dict(getattr(image, "bbox", None)),
    "local_path": str(destination_path) if download_success else None,
    "download_success": download_success,
}

```

Build a plain dict (like a DTO or anonymous type) capturing everything worth remembering:

- **index** — position in the document.
- **filename** — sanitized name.
- **category** — chart / figure / table / etc.
- **content\_type** — MIME type (image/png, image/jpeg).
- **bbox** — bounding box (x, y, width, height) converted from a custom object into a dict so it serializes to JSON. `object_to_dict` handles the `None` case gracefully.
- **local\_path** — where the file lives on disk, but only if download succeeded; otherwise `None` so downstream code doesn't chase a phantom path.
- **download\_success** — explicit flag for filtering / retry logic later.

```

image_records.append(image_record)

```

Add this record to the list. `List<T>.Add( )` equivalent.

```

print(f"{category or 'other'}: {filename} → {'saved' if download_success else 'failed'}")

```

Per-image status line. `category or 'other'` falls back to “other” if category is `None` or empty — Python's `or` returns the first truthy operand, similar to category ?? “other” in C#.

```

image_records_path = OUTPUT_DIR / "image_records.json"

```

Path for the manifest file. Note this is outside the `if` block — the manifest gets written even when no images existed (it'll just be []).

```

with image_records_path.open("w", encoding="utf-8") as file:

```

Open the file for writing with UTF-8 encoding. `with` is Python's `using` — auto-closes the handle even if an exception fires. UTF-8 matters for filenames or captions containing non-ASCII characters.

```

    json.dump(
        image_records,
        file,
        indent=2,
        ensure_ascii=False,
        default=str
    )

```

Serialize the list to JSON:

- **indent=2** — pretty-print, human-readable diffs.

- **ensure\_ascii=False** — keep Unicode characters as-is instead of escaping them to \uXXXX.
- **default=str** — fallback serializer: if the JSON encoder hits an object it doesn't know how to handle (a Path, datetime, custom class), call `str()` on it instead of throwing. A safety net.

```
print("Image records saved:", image_records_path)
```

Confirm the manifest write.

## Summary

The code processes the images LlamaCloud found in a parsed document by doing three things in a loop, then persisting the results:

1. **Sanitize and route** each image to a category-specific output folder.
2. **Download the bytes** via the presigned URL (which will expire) into that folder.
3. **Record metadata** — index, filename, category, MIME type, bounding box, local path, success flag — into a Python dict.

After the loop, it serializes the full list to `image_records.json` as a manifest. This gives you a clean separation: binary files sit on disk organized by category, while a single structured JSON file describes what was extracted, where each file lives, and whether the download succeeded. Downstream consumers (a RAG pipeline, review tool, fine-tuning dataset builder) read the manifest and dereference `local_path` to get the actual pixels.

The `getattr(..., None)` defensive reads and the `download_success` flag mean the code degrades gracefully — partial failures produce a valid manifest with `None` paths rather than crashing mid-batch.